Software Requirements Specification

For Smart Parking Platform

Version 6.0

Prepared by Ambar C. Pineda
$oftware corp.
October 28, 2026

Table of Contents

1. Introduction
    1.1. Document Purpose
    1.2. Product Scope
    1.3. Definitions, Acronyms, and Abbreviations
    1.4. References
    1.5. Document Overview
2. Product Overview
    2.1. Product Perspective
    2.2. Product Functions
    2.3. Product Constraints
    2.4. User Characteristics
    2.5. Assumptions and Dependencies
    2.6. Apportioning of Requirements
3. Requirements
    3.1. External Interfaces
    3.2. Functional
    3.3. Quality of Service
    3.4. Compliance
    3.5. Design and Implementation
    3.6. AI/ML
4. Verification
5. Appendixes

1. Introduction

Provides an overview of the document and orients the reader to the system being specified. Briefly summarize the SRS's purpose, product scope, intended audience, and how the document is organized. Do not include details here; reference the relevant sections instead.

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Smart Parking Platform. The primary audience for this document includes the engineering team (frontend, backend, DevOps), quality assurance (QA), project managers, and business stakeholders. It serves as the authoritative source for what the system must do to enable secure parking reservations, dynamic pricing, and operator management, ensuring all teams are aligned throughout the software lifecycle.

1.2 Product Scope

The Smart Parking Platform (Version 5.0) is a comprehensive mobile and web-based solution designed to streamline parking reservations, optimize lot utilization, and simplify facility management. Its primary capabilities include real-time spot availability, secure mobile checkout, dynamic surge pricing based on capacity, and digital QR-code access. The platform aims to reduce traffic congestion during major events and maximize revenue for parking operators by providing seamless integration between drivers, lot attendants, and city planners.

1.3 Definitions, Acronyms, and Abbreviations

Help readers understand specialized terms and notation by providing a glossary of domain terms, acronyms, and abbreviations used in the SRS. Tips: Include terms that impact interpretation of requirements (e.g., "user," "tenant," "near real-time"). Keep entries alphabetized and consistent across the document set.

Term	**Definition**
ADA	Americans with Disabilities Act (accessible parking compliance).
RBAC	Role-Based Access Control (permission levels for attendants vs. admins).
Dynamic Surge Pricing	Automated price adjustments based on lot capacity thresholds.
Tokenization	Replacing sensitive credit card details with non-sensitive digital identifiers.

1.4 References

Title: Project Charter: Smart Parking Platform (Version 5)

- **Author/Owner:** Ambar P. (Project Manager) and Charlie D. (Product Owner)
- **Version:** 1.0
- **Date:** September 11, 2026
- **Location/URL:** Section 5 (Appendixes) / Internal Project Repository
- **Classification:** Informative (Guidance)

1.5 Document Overview

Brief guide to the structure of the SRS so readers can quickly find what they need. Summarize what each major section covers (Product Overview, Requirements, Verification, Appendixes), note any document conventions, and mention how updates and revision history are managed.

2. Product Overview

Provides background and context influencing the product's requirements.

2.1 Product Perspective

Place the product within a larger ecosystem or lineage. Describe context and origin of the product, whether this is a new product, replacement, or member of a family. If part of a larger system, briefly explain relationships, external interfaces, and key dependencies. Include details on ownership, service level agreements (SLAs), and support models.

2.2 Product Functions

High-level summary of what the product enables users or systems to do. Provide a concise overview of the major functional areas/features. Defer detailed behaviors, data, and edge cases to Section 3. Tips: 5-10 bullets are often sufficient at this level, grouping related functions logically. Include a top-level data flow or use case diagram if helpful.

2.3 Product Constraints

Defines contextual limitations or conditions shaping design and implementation. Describe constraints such as mandated interfaces, technology stacks, regulatory obligations, QoS baselines, hardware limitations, AI/ML model families, and organizational policies.

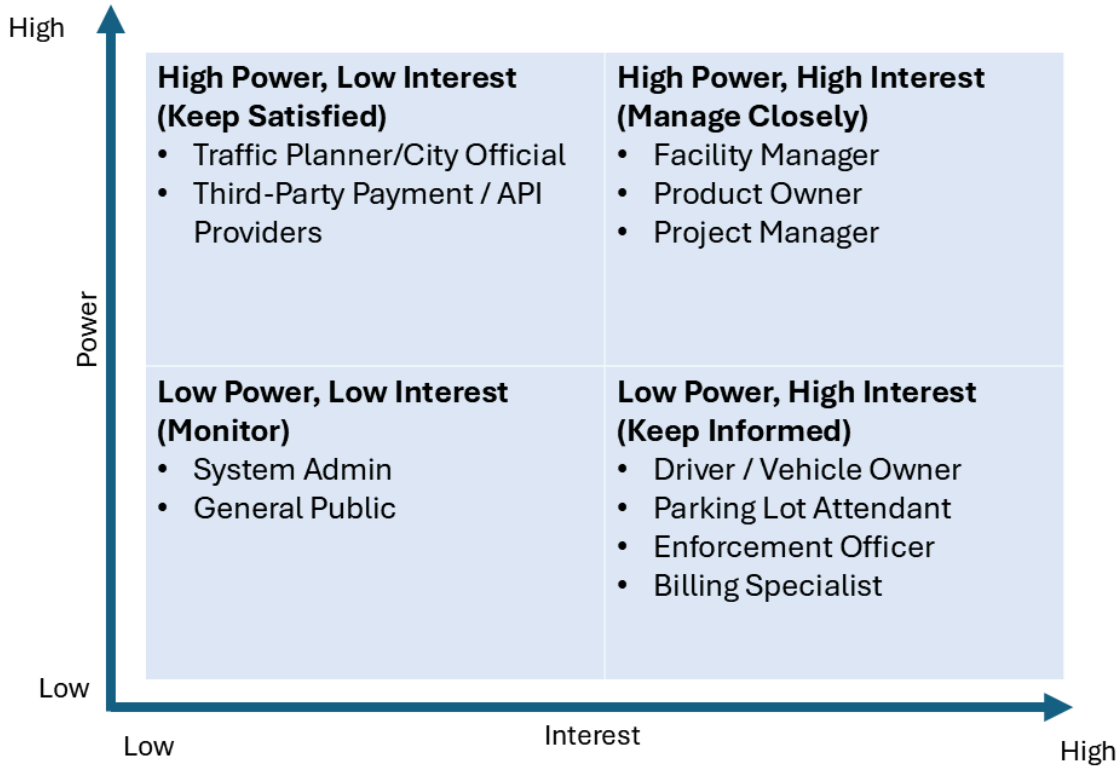
2.4 User Characteristics

Personas

The system will interact with the following primary user personas:

- **Driver / Vehicle Owner:** End-users seeking to find, reserve, and pay for parking via the mobile interface.
- **Facility Manager:** Administrative users who manage lot settings, monitor real-time dashboard capacity, and configure dynamic pricing rules.
- **Parking Lot Attendant:** On-site operators who scan customer QR codes, validate reservations, and manage manual check-ins using the operator app.
- **Enforcement Officer:** City or private personnel responsible for verifying license plates against active digital reservations.
- **Billing Specialist / Finance Manager:** Back-office users who handle automated partial refunds, daily settlement reports, and financial tracking.
- **Traffic Planner / City Official:** External stakeholders utilizing aggregated platform heat maps to optimize road closures and direct city traffic flow.
- **System Admin:** IT personnel overseeing access control, database health, and API integrations.

Stakeholder Analysis (Power/Interest Grid)



Quadrant	Stakeholder Roles	Engagement Strategy
High Power, High Interest (Manage Closely)	**Facility Manager:** Controls lot settings, dynamic pricing, and monitors dashboards. **Product Owner (Charlie D.):** Accountable for requirements and overall product value. **Project Manager (Ambar P.):** Responsible for schedule, budget, and risk mitigation.	Require daily engagement, direct involvement in scope approvals, and real-time visibility into project metrics and system dashboards.
High Power, Low Interest	**Traffic Planner / City**	Provide aggregated heat map

(Keep Satisfied)	**Official:** External stakeholder using platform data for municipal routing. **Third-Party Payment / API Providers:** Providers of tokenized payments and mapping endpoints critical to platform function.	data and maintain strict adherence to API security rules to prevent credential exposure or service interruption.
Low Power, High Interest (Keep Informed)	**Driver / Vehicle Owner:** End-users searching, booking, and paying for spots. **Parking Lot Attendant:** Scans QR codes and manages live check-ins. **Enforcement Officer:** Validates license plates in real time. **Billing Specialist:** Manages partial refunds and daily settlements.	Ensure adequate training (e.g., Operator Training Mode), clear UI instructions, and timely system alerts regarding reservations or errors.
Low Power, Low Interest (Monitor)	**System Admin:** Manages databases, health, and user access. *(Note: Becomes High Interest during incidents)* **General Public:** Non-app users impacted by city traffic routing.	Rely on automated PagerDuty alerts and synthetic checkout latency monitoring to ensure system stability without requiring constant manual intervention.

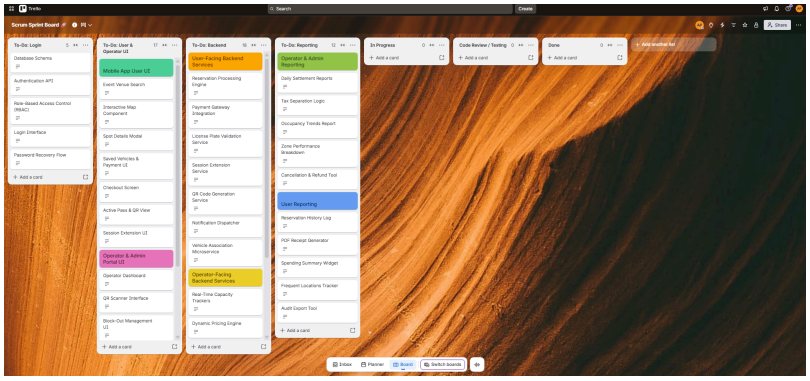
2.5 Assumptions and Dependencies

External assumed factors or conditions, as opposed to known facts, that the project relies on. List assumptions about environment, hardware, usage patterns, third-party components/services, and organizational support.

ID	Category	Description	Probability	Impact	Owner	Response Strategy	Notes	Status
TR-01	Technical	API latency exceeds 500ms causing checkout timeouts	Medium	High	Sysadmin	Mitigate	Automated alerts needed	Open
TR-02	Technical	QR Code scanner interface fails for lot attendants	Low	High	Backend Team	Mitigate	Fallback to manual entry	In Progress
TR-03	Technical	Tokenized payment webhook fails to notify app	Low	High	3rd Party Provider	Transfer	Monitor SLA	In Progress
TR-04	Technical	Database struggles with dynamic surge pricing load	Medium	Medium	DBA	Mitigate	Optimize queries	Closed
SR-01	Schedule	Frontend/Backend integration takes longer than the allocated 10 days	High	Medium	Project Manager	Mitigate	Daily standups	In Progress
SR-02	Schedule	UAT sign-off delayed by stakeholders	Medium	High	Project Manager	Mitigate	Schedule early check-ins	Closed
SR-03	Schedule	App Store/Play Store deployment delayed	Low	Medium	Release Manager	Accept	Start review process early	In Progress
SR-04	Schedule	Cloud pipeline setup blocks development	Low	High	DevOps	Avoid	Pre-configure environment	Open
FR-01	Financial	Platform processing fees cut into margins	High	Low	Finance Manager	Accept	Build into pricing model	Open
FR-02	Financial	Tax separation logic error causes incorrect remittances	Low	High	Accountant	Avoid	Thorough testing	Closed
FR-03	Financial	Partial refunds process fails, requiring manual labor	Medium	Medium	Billing Specialist	Mitigate	Staged rollout	In Progress
FR-04	Financial	Extended backend dev time increases budget	Medium	High	Project Manager	Mitigate	Track hours closely	Closed
PR-01	People	Key frontend developer leaves during 20-day dev block	Low	High	HR / PM	Mitigate	Cross-training	In Progress
PR-02	People	Lot attendants refuse to adopt new QR system	Medium	Medium	Facility Manager	Mitigate	Training sessions	Closed
PR-03	People	City enforcement officers misinterpret heat map data	Low	Low	City Official	Accept	Provide manual	Open
PR-04	People	Sysadmin misses automated alerts	Low	High	IT Manager	Mitigate	PagerDuty integration	Open

2.6 Apportioning of Requirements

Allocation of requirements across components or increments. Map major requirements to subsystems, services, or releases/iterations. Use a cross-reference table to show allocation and to clearly identify deferred requirements.



Task	Developers	Project Manager	Quality Assurance	UX	DevOps	Business Analysts	Product Owner	Subject Matter Expert
Requirements	I	A	I	C	I	R	A	C
Design	C	A	I	R	I	C	A	C
Implementation	R	A	I	C	C	I	I	C
Testing	C	A	R	I	I	C	I	I
Deployment	C	A	C	I	R	I	I	I

3. Requirements

This section specifies verifiable requirements of the software product to enable design and testing. State requirements to a level of detail sufficient for design and verification.

3.1 External Interfaces

Specifies all external inputs and outputs, covering both required and provided interfaces. Provide interface definitions sufficient for implementation and test.

3.1.1 User Interfaces

Describes how users interact with the system at a logical level.

3.1.2 Hardware Interfaces

Details interactions with physical devices and platforms.

3.1.3 Software Interfaces

Defines integrations with other software components and services.

3.2 Functional

Specifies the externally observable behaviors and functions the software shall provide.

ID: REQ-FUNC-001

- **Title:** Secure Driver Authentication
- **Statement:** The system shall provide a secure login mechanism allowing a Driver/Vehicle Owner to save preferred payment methods, license plate numbers, vehicle details, and view reservation history.
- **Rationale:** Required to fulfill User Story 1 and provide a personalized, secure user experience.
- **Acceptance Criteria:** A user can successfully create an account, log in securely, and save or retrieve profile data.
- **Verification Method:** Test

ID: REQ-FUNC-002

- **Title:** Real-Time Parking Availability
- **Statement:** The system shall display real-time parking availability to the Driver/Vehicle Owner based on current lot capacity and active reservations.
- **Rationale:** Fulfills User Story 2.
- **Acceptance Criteria:** Available spot counts update instantly when a new reservation is confirmed or a vehicle is checked in.
- **Verification Method:** Test / Demonstration

ID: REQ-FUNC-003

- **Title:** Cross-Lot Price Comparison

- **Statement:** The system shall allow the Driver/Vehicle Owner to view and compare dynamic pricing across different parking garages in a selected geographical area.
- **Rationale:** Fulfills User Story 3.
- **Acceptance Criteria:** The UI displays a list or map of garages with their respective current active prices clearly visible.
- **Verification Method:** Test

ID: REQ-FUNC-004

- **Title:** Garage Amenities and Smart Tags Display
- **Statement:** The system shall display garage amenities (including ADA parking availability and facility security features) and smart tags (such as "shortest walk" and "best value") for each parking facility.
- **Rationale:** Fulfills User Story 4, enabling users to evaluate facilities based on accessibility, security, and convenience.
- **Acceptance Criteria:** Selecting a facility reveals an amenities breakdown showing ADA compliance, security details, and applicable smart tags.
- **Verification Method:** Test

ID: REQ-FUNC-005

- **Title:** Multi-Channel Expiration Notifications
- **Statement:** The system shall send notifications ahead of reservation expiration via in-app push notifications for app users, and via SMS or email for users without the app.
- **Rationale:** Fulfills User Story 5 to alert drivers prior to session end times across preferred channels.
- **Acceptance Criteria:** Automated alerts trigger at a configured pre-expiration threshold and deliver reliably via push, SMS, or email according to user state and preference.
- **Verification Method:** Test

ID: REQ-FUNC-006

- **Title:** Event-Based Venue Search
- **Statement:** The system shall enable drivers to search for specific events and return a list of nearby available parking garages.
- **Rationale:** Fulfills User Story 6, allowing event-goers to quickly locate proximate parking.
- **Acceptance Criteria:** Querying an event name or venue returns valid, geographically proximity-sorted parking garage listings.
- **Verification Method:** Test

ID: REQ-FUNC-007

- **Title:** Amenity-Based Garage Filtering

- **Statement:** The system shall allow drivers to filter search results by specific amenities, including valet parking and EV charging stations.
- **Rationale:** Fulfills User Story 7 to assist drivers in finding facilities tailored to their vehicle or service preferences.
- **Acceptance Criteria:** Applying filters updates the search results to display only garages that meet all selected amenity criteria.
- **Verification Method:** Test

ID: REQ-FUNC-008

- **Title:** Facility Address and Navigation Details
- **Statement:** The system shall provide the complete physical address and navigation destination details of the reserved parking facility to the driver.
- **Rationale:** Fulfills User Story 8 so drivers can navigate accurately to their reserved location.
- **Acceptance Criteria:** Facility profiles and reservation confirmations clearly display the street address and provide routing integration points.
- **Verification Method:** Inspection / Test

ID: REQ-FUNC-009

- **Title:** Mobile Parking Pass Delivery and Digital Wallet Integration
- **Statement:** The system shall provide a digital parking pass accessible within the mobile application and deliverable via email/SMS, supporting one-click addition to Apple Wallet and Google Wallet.
- **Rationale:** Fulfills User Story 9 to enable fast, offline, and native mobile wallet access to passes.
- **Acceptance Criteria:** The driver can view passes in-app and successfully export the generated pass credential directly into Apple Wallet or Google Wallet.
- **Verification Method:** Test / Demonstration

ID: REQ-FUNC-010

- **Title:** Interactive Parking Map Interface
- **Statement:** The system shall render an interactive map showing available parking garages, rates, and occupancy indicators.
- **Rationale:** Fulfills User Story 10 to provide spatial context and intuitive geographic selection.
- **Acceptance Criteria:** The map renders pins with garage metadata, responds to zooming/panning, and updates upon filtering or searching.
- **Verification Method:** Test / Demonstration

ID: REQ-FUNC-011

- **Title:** Real-Time Reservation Time Extension

- **Statement:** The system shall allow drivers to extend the duration of an active parking reservation directly from the mobile application, subject to spot availability.
- **Rationale:** Fulfills User Story 11 to accommodate running late without incurring penalties or re-booking.
- **Acceptance Criteria:** The user can select an active pass, choose an additional time block, complete payment, and receive updated expiration timestamps.
- **Verification Method:** Test

ID: REQ-FUNC-012

- **Title:** Facility Manager Capacity Dashboard and Dynamic Surge Pricing
- **Statement:** The system shall provide facility managers with a real-time capacity dashboard displaying reserved vs. drive-up occupancy, and automatically trigger or enable dynamic surge pricing when total lot occupancy reaches 80%.
- **Rationale:** Fulfills User Story 12 to optimize lot yield and manage demand fluctuations.
- **Acceptance Criteria:** The dashboard updates live counts; reaching 80% threshold highlights the facility and engages surge rate adjustments.
- **Verification Method:** Test / Demonstration

ID: REQ-FUNC-013

- **Title:** Lot Attendant QR Code Check-In Scanner
- **Statement:** The system shall provide an operator interface capable of scanning a customer's digital parking pass QR code to instantly validate reservations without manual paper cross-referencing.
- **Rationale:** Fulfills User Story 13 to streamline vehicle entry and reduce ingress bottlenecking.
- **Acceptance Criteria:** Scanning a valid pass returns an instant confirmation check; scanning an expired or duplicate pass returns an immediate error state.
- **Verification Method:** Test

ID: REQ-FUNC-014

- **Title:** Enforcement Officer License Plate Validation
- **Statement:** The system shall provide an enforcement interface that validates license plate numbers against active reservation databases in real time.
- **Rationale:** Fulfills User Story 14 to enable rapid field compliance checks.
- **Acceptance Criteria:** Entering or capturing a license plate returns current reservation status, expiration time, and validity within 1 second.
- **Verification Method:** Test

ID: REQ-FUNC-015

- **Title:** Driver Self-Service Cancellation and Policy Display

- **Statement:** The system shall allow drivers to cancel reservations through the self-service interface and display the applicable cancellation terms, deadlines, and fee structures prior to confirmation.
- **Rationale:** Fulfills User Story 15 to ensure transparent booking management.
- **Acceptance Criteria:** Drivers can trigger cancellation; the system displays policy terms, requires confirmation, and updates spot status to available.
- **Verification Method:** Test

ID: REQ-FUNC-016

- **Title:** Automated Partial Refund Processing
- **Statement:** The system shall execute automated partial refund transactions according to business rules when events or driver reservations are canceled.
- **Rationale:** Fulfills User Story 16, eliminating manual intervention for high-volume cancellation events.
- **Acceptance Criteria:** Qualifying cancellations trigger automated payment gateway refund calls and generate settlement records.
- **Verification Method:** Test

ID: REQ-FUNC-017

- **Title:** System Administrator Account Reset and User Support
- **Statement:** The system shall provide system administrators with administrative utilities to manage, reset, and resolve user login, authentication, and credential issues.
- **Rationale:** Fulfills User Story 17 to restore customer and operator account access.
- **Acceptance Criteria:** Admin console allows account unlocking, password reset dispatch, and session invalidation.
- **Verification Method:** Demonstration / Test

ID: REQ-FUNC-018

- **Title:** Municipal Traffic Planner Heat Map Analytics
- **Statement:** The system shall aggregate parking utilization data and render visual heat maps during major events to assist traffic planners in routing and municipal road closure decisions.
- **Rationale:** Fulfills User Story 18 to optimize urban traffic flow and minimize congestion.
- **Acceptance Criteria:** Aggregated event occupancy can be viewed as density overlays across designated city traffic zones.
- **Verification Method:** Demonstration

ID: REQ-FUNC-019

- **Title:** Daily Financial Settlement and Reconciliation Reports

- **Statement:** The system shall generate automated daily settlement reports detailing gross revenue, itemized third-party processing fee deductions, and net amounts for bank deposit reconciliation.
- **Rationale:** Fulfills User Story 19 to streamline accounting and financial reconciliation.
- **Acceptance Criteria:** Daily reports compile at 23:59:59 each day, accurately breaking down gross receipts, fees, and net payouts.
- **Verification Method:** Test / Inspection

ID: REQ-SEC-020
- **Title:** Tokenized Payment Integration
- **Statement:** The system shall process digital payments using third-party tokenization (Apple Pay, Google Pay, and tokenized payment gateways) without storing raw Primary Account Numbers (PAN) or CVVs on internal servers.
- **Rationale:** Fulfills User Story 20, satisfying PCI-DSS compliance and minimizing data breach liability.
- **Acceptance Criteria:** Network traces and database schemas confirm only non-sensitive tokens and transaction references are retained.
- **Verification Method:** Inspection / Test

ID: REQ-SEC-021
- **Title:** Mapping API Token Protection
- **Statement:** The system shall securely manage and restrict Google Maps and Apple Maps API tokens to authorized backend proxies and authenticated domains to protect credential integrity and user data.
- **Rationale:** Fulfills User Story 21 to prevent unauthorized API consumption and credential exposure.
- **Acceptance Criteria:** API keys are stored in secure environment secrets, restricted by HTTP referrer/IP, and obfuscated from public client-side access.
- **Verification Method:** Inspection

ID: REQ-SEC-022
- **Title:** Role-Based Access Control (RBAC) for Dashboards
- **Statement:** The system shall enforce Role-Based Access Control (RBAC) across administrative and operator dashboards, preventing unauthorized roles (such as lot attendants) from viewing or modifying facility pricing configurations.
- **Rationale:** Fulfills User Story 22 to prevent accidental or unauthorized operational alterations.
- **Acceptance Criteria:** Unauthorized role requests to pricing or admin endpoints receive HTTP 403 Forbidden; UI elements for pricing are hidden from non-admin accounts.
- **Verification Method:** Test

ID: REQ-FUNC-023

- **Title:** Manual Reservation Code Fallback Entry
- **Statement:** The system shall provide an attendant-facing manual alphanumeric reservation ID entry field as a fallback mechanism when camera hardware or QR scanning fails.
- **Rationale:** Fulfills User Story 23 to prevent vehicle entry blockages caused by scanner malfunctions.
  Acceptance Criteria: Attendant can enter an 8-to-12 character reservation ID manually and receive equivalent verification status as an optical scan.
- **Verification Method:** Test

ID: REQ-FUNC-024

- **Title:** Operator Training Mode Simulation
- **Statement:** The system shall provide an isolated "training mode" toggle in the operator mobile application that allows staff to scan mock QR codes without modifying live lot capacity, occupancy numbers, or revenue ledgers.
- **Rationale:** Fulfills User Story 24 to enable attendant onboarding without corrupting operational data.
- **Acceptance Criteria:** In training mode, scan validation responses display visual feedback while database write operations to live inventory tables are blocked.
- **Verification Method:** Test

ID: REQ-OBS-025

- **Title:** PagerDuty Escalation for Checkout Latency
- **Statement:** The system shall monitor API checkout endpoints and immediately dispatch automated high-urgency alerts via PagerDuty whenever API latency exceeds 500ms or error rates spike.
- **Rationale:** Fulfills User Story 25 and mitigates Risk TR-01 to ensure operational responsiveness during checkout bottlenecks.
- **Acceptance Criteria:** Synthetic or production checkout latencies > 500ms trigger a PagerDuty incident within 60 seconds.
- **Verification Method:** Test

ID: REQ-PERF-026

- **Title:** Database Read-Replicas for Surge Availability Queries
- **Statement:** The system database architecture shall deploy read-replicas dedicated to serving read-heavy parking availability queries to isolate the primary database from surge traffic overloads.
- **Rationale:** Fulfills User Story 26 and mitigates Risk TR-04, preventing master database crashes during dynamic pricing events.

- **Acceptance Criteria:** Availability search queries are routed strictly to read-replicas, maintaining master DB CPU utilization under 60% during peak loads.
- **Verification Method:** Test / Analysis

ID: REQ-FUNC-027

- **Title:** Batch Refund Fallback Dashboard
- **Statement:** The system shall provide a dedicated fallback utility in the billing dashboard allowing specialists to batch-reprocess failed refund transactions resulting from webhook drops or gateway timeouts.
- **Rationale:** Fulfills User Story 27 and mitigates Risk FR-03, eliminating repetitive manual individual entries.
- **Acceptance Criteria:** Failed refund items can be filtered, selected in bulk, and resubmitted to the payment gateway with unified status reporting.
- **Verification Method:** Test

ID: REQ-FUNC-028

- **Title:** Dynamic Pricing Margin Calculator Guardrail
- **Statement:** The dynamic pricing engine shall incorporate a real-time margin calculator that restricts rate discounts to ensure projected net prices never fall below aggregate third-party platform processing fees.
- **Rationale:** Fulfills User Story 28 and mitigates Risk FR-01, preventing negative operating margins during off-peak discounts.
- **Acceptance Criteria:** The system rejects any automated or manual pricing rule that results in an effective net margin <=$0.00 after processing fee deductions.
- **Verification Method:** Test

ID: REQ-FUNC-029

- **Title:** Heat Map Tooltips, Legend, and Municipal Data Context
- **Statement:** The system utilization heat map shall embed contextual tooltips, occupancy thresholds, and a standardized color legend to prevent misinterpretation during municipal planning.
- **Rationale:** Fulfills User Story 29 and mitigates Risk PR-03, ensuring clear interpretation by city officials.
- **Acceptance Criteria:** Hovering over heat map zones renders exact occupancy percentages, active spot counts, and timestamp ranges accompanied by a clear explanatory key.
- **Verification Method:** Demonstration / Inspection

3.3 Quality of Service

Quality attributes that constrain or qualify functional behavior. Use specific metrics, ranges, and conditions.

3.3.1 Performance

Response time, throughput, and resource usage expectations.

ID: REQ-PERF-001

- **Title:** Maximum API Latency for Checkout
- **Statement:** The payment integration and checkout APIs shall process transactions and respond in under 500ms during peak load.
- **Rationale:** Prevents checkout timeouts and addresses Risk TR-01.
- **Acceptance Criteria:** The 95th percentile latency remains below 500ms during simulated peak traffic load tests.
- **Verification Method:** Test (Load Testing)

3.3.2 Security

Defines the protection of data, identities, and operations. Define authentication, authorization, data protection (in transit/at rest), auditing, and privacy requirements. Address abuse/misuse and external attacks (e.g., injection, data exfiltration, or service compromise), and include secure defaults and incident response requirements.

3.3.3 Reliability

Ability to consistently perform as specified.

3.3.4 Availability

System uptime and readiness to deliver service.

3.3.5 Observability

Ability to understand system state and behavior in production through telemetry.

3.4 Compliance

Requirements derived to satisfy external standards, regulations, or contracts.

3.5 Design and Implementation

Constraints or mandates affecting how the solution is designed, deployed, and maintained.

3.5.1 Installation

Ensures the software runs smoothly in its target environments.

3.5.2 Build and Delivery

Defines the controls for building, packaging, and delivering software artifacts to ensure integrity, traceability, and reproducibility.

3.5.3 Distribution

Addresses geographically or organizationally distributed deployments, data, and devices.

3.5.4 Maintainability

Attributes that make the software easier to modify, fix, and evolve.

3.5.5 Reusability

Encourages leveraging components across products or contexts when appropriate.

3.5.6 Portability

Ability to run on multiple platforms or environments with minimal changes.

3.5.7 Cost

Financial considerations or cost targets.

Resource Plan

Role	Name	FTE %	Estimated Hours	Hourly Rate	Cost
Project Manager	Ambar P.	50%	320	$100	$32,000
Business Analysts	Bob M.	30%	160	$80	$12,800
Product Owner	Charlie D.	25%	120	$110	$13,200
UX	Diana K.	50%	240	$90	$21,600
Developers	Evan & Fiona	100%	960	$85	$81,600
Quality Assurance	George T.	50%	240	$75	$18,000
DevOps	Hannah P.	30%	160	$95	$15,200
Subject Matter Expert	Ian V.	10%	40	$120	$4,800
Total Labor			**2240**	$755.00	**$199,200.00**

Cost Plan

Item	Category	Qty/Months	Unit Cost	Duration (months)	Total	Notes
Project Labor	Labor	1	$199,200	4	$199,200	Full labor cost from Resource Plan covering the

						4-month timeline.
Cloud Hosting & DB	Infrastructure	1	$2,000	4	$8,000	Server provisioning, storage setup, and primary database with read-replicas.
Mapping & Payment APIs	Licenses/Tools	1	$600	4	$2,400	Required for Google Maps/Apple Maps API tokens and tokenized payment integrations.
Dev & Incident Tools	Licenses/Tools	1	$400	4	$1,600	CI/CD automation software and PagerDuty integration for latency alerts.
Subtotal					**$211,200**	
Contingency Reserve	Reserve	10%			$21,120	10% contingency applied to the subtotal to cover unbudgeted overruns.
Total Project Cost					**$232,320**	

Cashflow Table

Month	Labor	Infra	Licenses	Contingency	Total
September 2026	$30,000	$2,000	$1,000	$3,300	$36,300
October 2026	$60,000	$2,000	$1,000	$6,300	$69,300
November 2026	$80,000	$2,000	$1,000	$8,300	$91,300
December 2026	$29,200	$2,000	$1,000	$3,220	$35,420
Total	**$199,200.00**	**$8,000.00**	**$4,000.00**	**$21,120.00**	**$232,320.00**

3.5.8 Deadline

Schedule expectations that affect scope and prioritization. Specify key milestones, delivery dates, or phases/increments. Indicate dependencies between milestones and required readiness criteria.

WBS

1. General & Project Management

1.1 Project Initiation

* Project Charter & Stakeholder Register

1.2 Project Planning

* Project Schedule & Resource Allocation

1.3 Project Monitoring & Plan Validation

* Status Reports & Risk Log

2. Analysis & Detailed Design

2.1 Scope & Requirements

* User Stories & Scope Diagram

2.2 System Architecture

* Data Flow Diagrams (DFD)

2.3 UI/UX Design & Usability Testing

* Wireframes & Interactive Prototypes

3. Database & Infrastructure

3.1 Database Setup & Schema Validation

* Schema creation for users, spots, and transactions

3.2 Cloud Pipeline & Connection Testing

* Server provisioning & storage setup

* CI/CD automation scripts

4. Frontend & Backend Development

4.1 Backend

* Authentication – User registration & secure login

* Inventory Management – Parking spot availability & CRUD operations

* Payment Gateway – Secure checkout & refund processing

4.2 Frontend

* Homepage & Navigation – Search interface & interactive map

* Booking Flow – Spot selection & cart actions

* User Dashboard – Reservation history & account settings

5. Final Validation & UAT

5.1 User Acceptance Testing (UAT)

* Beta testing & stakeholder sign-off

5.2 Performance & Security Testing

* Load testing & vulnerability scanning

6. Project Closure

6.1 Final Deployment

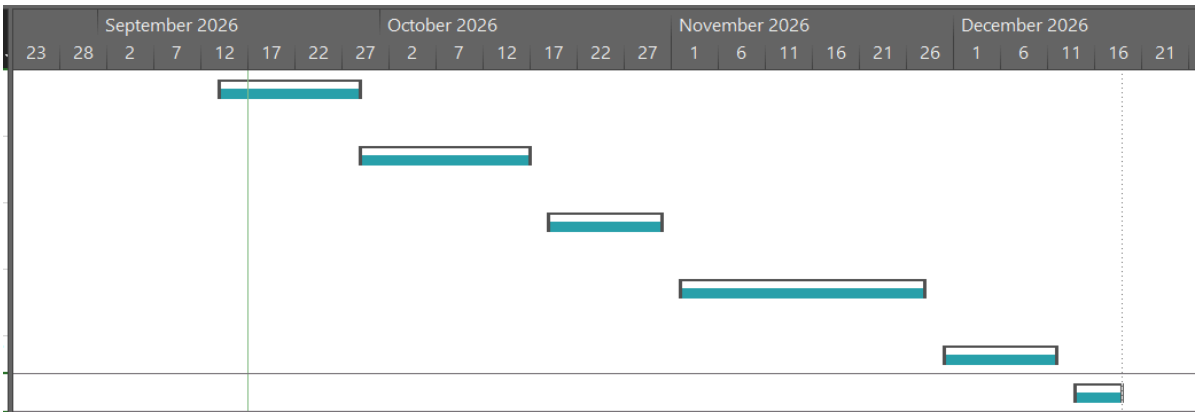
* Production launch & App Store/Play Store release

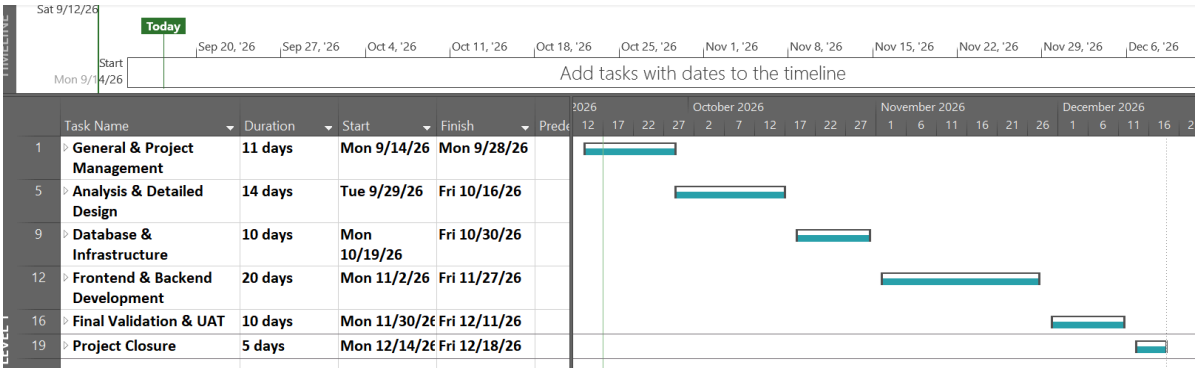
6.2 Handoff & Training

* System documentation & support guides

6.3 Project Review

* Lessons learned & final project sign-off





3.5.9 Proof of Concept

Validates feasibility and de-risks critical assumptions before full-scale delivery.

3.5.10 Change Management

Controls how changes are introduced and communicated.

3.6 AI/ML

This section defines requirements unique to systems incorporating machine learning or data-driven components at their core.

4. Verification

Describes how each requirement will be verified to provide objective evidence of compliance. Outline verification methods (test, canary metrics, analysis, inspection, demonstration) and test evidence preferably in a matrix paralleling Section 3. Consider adding environment details, tools, and test data requirements.

| Requireme nt ID | Verificatio n Method | Test/Artifa ct Link | Status | Evidence | |
REQ-FUNC -001 | test | tests/UC01 .md | Passed | reports/tuc 01.html | |
REQ-SEC-0 03 | analysis | threat-mod el.md | WIP | |

5. Appendixes

Optional supporting material that aids understanding without being normative.

5.1 AI Use

Homework 1

1. What tool was used:

Gemini

2. What it was used for:

I used Gemini to help brainstorm user stories for different personas using ParkMobile and SpotHero.

3. The prompt(s) used:

Prompt 1:

https://app.parkmobile.io/venues/atlanta-symphony-hall/events/297349

https://spothero.com/search?kind=event&id=1375435&view=dl

Look at these different parking sites and help me brainstorm user stories for Parking facility operators City transportation departments System administrators Finance and billing personnel Mobile application users Third-party payment providers

4. What I kept vs. What I changed
    - **Kept**: I kept some of the backend/admin stories (12,13,18,19,20,22)
    - **Modified**: I adapted the AI's automated partial refunds story into my billing specialist story (16)
    - **Modified:** I came up with license plate checking, however, after looking at the ai response I changed the user from "parking facilities operator" to "enforcement officer" as I found it more fitting
    - **New:** I wrote stories 1-11, 17, 21. Story 21 is inspired by Story 20's token and these references: https://techcrunch.com/2024/10/08/google-partners-with-spothero-convenient-parking-reservations-maps-search/ https://developers.google.com/maps/api-security-best-practices

Homework 2

1. What tool was used:

Gemini

2. What it was used for:

I used Gemini to help generate a three-level WBS for the project. I also used it to see how long tasks should last and to include QA testing for every phase.

3. The prompt(s) used:

- "Create a starting base for a work break down structure that is no more than three levels for a website and app that helps with parking

reservations. Here is an example of what the format should look like:"

- "Let's add more in this order: 1. General & Project Management (think project initiation) 2. Analysis & Detailed design (scope diagram, DFD, User stories) 3. Frontend 4. Backend 5.Database & Infrastructure 6. Testing 7. Project Closure"

- "Within this srs template, where can I place my user stories and my work break down structure"

- "For the work breakdown structure what is a good timeline for a project of this type"

- "Can you give me the amount of days it will take for each task to take. For example I have the project started on 14th of september and I have it so phase one lasts between september 14 to september 28 making it 11 days on project."

- "Is there a way to have testing in every phase?"

4. What I kept vs what I changed:
    - **Kept:** I kept the structure, task names, and duration estimates exactly as was generated. I typed it into Microsoft Project to create my Gantt chart.
    - **Modified**: I adjusted some of the task names within Microsoft Project for formatting purposes. I also set up the project start date as September 14, 2026, in MS Project while keeping the AI's standard working day calculations.

Homework 3

1. What tool was used:

Gemini

2. What it was used for:

I used Gemini to help brainstorm and structure a Sprint 1 backlog. I used it to create specific c task cards divided into categories. I also used it to figure out a color-coded tagging that will be implemented later on.

3. The prompt(s) used:

- "Create a scrum board for the sprint 1 backlog. I am using Trello. Include 5 items for the login, 15 items for the UI for both the operator and user, 10 items for reporting for operator and user."
- "Should these be the column headings? I do not want this to overpopulate the to-do section"
- "Should I create a different ones for In Progress and Done columns? also can you tailor the items for a parking reservation app and website please. Here are the user stories to better tailor the items"
- "I meant have columns like In progress: Login, In progress: User & Operator UI, etc."
- "I forgot but can you re do the items and add 15 items for backend processes for operator and user"
- "There are colored labels in Trello that I can use which items can have what labels and what labels should I add"
- "I am creating a scrum board on Jira for a parking reservation app. The backlog for the parking app should be login (min 5 items), UI (15 items), the backend processes for operator and use (min 15 items), reporting for operator and user (min 10 items). I also need a sprint 1 backlog on this same board"

4. What I kept vs. What I changed
    - **Verbatim / Kept:** I kept the task breakdown for Login (5 items), UI (15 items), Reporting (10 items), and Backend Processes (15 items) generated by the AI after verifying they matched my project's requirements. I also used the recommended Trello column structure (categorized To-Do columns, a unified In Progress section, and Code Review/Testing) and the persona-based color labeling system (e.g., Green for Mobile User, Blue for Operator, Red for Backend) exactly as suggested.
    - **Modified / Changed:** While one of my prompts asked for Jira formatting, I ultimately applied it to Trello. I manually set up the Trello board, created the cards, and applied the labels myself based on the AI's blueprint.

Homework 4

1. What tool was used:

Gemini

2. What it was used for:

I used Gemini to help brainstorm a 16-entry Risk Register for the Smart Parking Platform, covering technical, schedule, financial, and people risks. I also used it to

help guide me on conditional formatting Google Sheets. Finally, I used it to create new user stories based on these newly identified project risks.

3. The prompt(s) used:
    - "Look at these four pdf as context for what you need to do: Using excel I need to create a Risk Register structured as a list with at least 10 entries. It must cover four main categories: technical, schedule, financial, and people. For each risk entry, provide the following specific details: - ID and Description - Probability and Impact - Owner and Response Strategy (specify whether to Avoid, Mitigate, Transfer, or Accept) - Status/Notes" There must also be at least 4 risks for each category."
    - "How can I get it so that any risk labeled as high is red, medium is orange, and low is green?"
    - "Should response strategy be color coded as well?"
    - "Can you add a ranking like as seen on the screenshot? The ranking should rank based on the impact. Furthermore, I would like for the spreadsheet to be formatted so that others are able to see select rows such categories and owners."
    - "2. Add the "Ranking" Column... I wanted it as a numbered ranking instead"
    - "No each row should have their own ranking. Every row with a impact high should have a greater ranking than medium, medium should have a higher ranking than low"
    - "- I would like to separate these two: For status I would like to have these categories: Open / Identified, In Progress / Under Review, Mitigated/Controlled, Occurred / Realized, and Closed / Retired. Please look at the list and just randomly apply them when applicable."
    - "based on these new risks what are new user stories that can occur"
    - "Reformat so it is more similar to this format"
4. What I kept vs. what I changed:
    - **Verbatim / Kept:** I kept the 16 specific risks generated for the Risk Register, including their IDs, descriptions, and response strategies, which I inputted into my spreadsheet.
    - **Modified / Changed:** I manually applied the suggested Google Sheets conditional formatting and dropdown data validation rules myself. I also kept the newly generated user stories (numbers 23 through 29) exactly, but I changed the formatting so it is more uniformed with the previous user stories.

Homework 5

1. **What tool was used:** Google Gemini

2. **What it was used for:** I generated a RACI matrix, a bottom-up Resource Plan, a Cost Plan with a 10% contingency reserve, a Cashflow Table, and a recommended technology stack for the Smart Parking Platform semester project.
3. **The prompt(s) used:**
    - "Use these files as context and using a bottom-up estimation approach generate the following 1. RACI Matrix Create a RACI matrix (Responsible, Accountable, Consulted, Informed). Include the following roles: developers, project manager, Quality assurance UX, DevOPs, Business analysts, product owner, and Subject matter expert. Map them against the following tasks: Requirements, Design, Implementation, Testing, and Deployment. 2. Resource Plan Create a bottom-up estimation table with the following columns: Role, Name, FTE %, Estimated Hours, Hourly Rate, and Cost. - use the roles listed above. - Assign reasonable hourly rates and estimate realistic hours 3. Cost Plan Create a table with the columns: Item, Category, Qty/Months, Unit Cost, Duration (months), Total, and Notes. Include... total Labor costs from the Resource Plan. - Infrastructure/Cloud hosting costs. - Licenses/Tools costs. Calculate and add a [Insert %, e.g., 10%] Contingency reserve applied to the subtotal before contingency. 4. Cashflow Table Create a cashflow table outlining the spending timeline with the columns: Month, Labor, Infra, Licenses, Contingency, and Total. Distribute the costs from the Cost Plan logically across the project's months. Ensure all calculations are accurate, come up with the appropriate tech for this project as well"
    - "can you make it so that i can copy the cost plan table"
    - "no like create a table and let me copy it"
4. What I kept vs. what I changed:
    - **Verbatim / Kept:** I kept the financial estimates, labor hour calculations, and hourly rates generated by the AI for the Resource Plan and Cost Plan. I also kept the Cashflow Table's monthly distributions and the role assignments mapping in the RACI matrix.
    - **Modified**: I transferred the tables that were generated onto google sheets and color coded the RACI table through conditional formatting. I also formatted the tables for Resource Plan, Cost Plan, and Cashflow Table.

Homework 6

1. **What tool was used:** Google Gemini
2. **What it was used for:** I used ai to identify discrepancies in my project files, format my user stories into formal Software Requirements Specification (SRS) statements (REQ-FUNC, REQ-SEC, REQ-PERF, etc.), and I also used it to get advice on where to place my assignments and the AI disclosure within the final document.

3. **The prompt(s) used:**
    - "All these files are related to the parking reservation app. Tell me what updates I should do on these files so that there are no discrepancies. Once done Follow the SRS template and place each of these files under the correct sections. For files that are pictures/google sheets write in bold red [INSERT <NAME OF TASK> HERE]. Do not change any of the wording from the files copy it exactly."
    - "can you make the document into a google doc instead?"
    - "[Pasted SRS template text for section 1.3] Whats the purpose of this"
    - "What else can be currently added to the srs document?"
    - "Do everything"
    - "[Pasted examples of REQ-FUNC-001, REQ-FUNC-002, REQ-FUNC-003] do the rest of the user stories in this format"
    - "Where should ai_use be on the srs document?"
    - "Where do i place all four tables in the srs?"
4. **What I kept vs. what I changed:**
    - **Verbatim / Kept:** I kept the requirement statements (REQ-FUNC-004 through REQ-FUNC-029 and the various REQ-SEC/REQ-PERF constraints) as the AI formatted them from my user stories. I also kept the new definitions provided for the Glossary.
    - **Modified:** I manually modified Software-Corp SRS, making sure all the tables, Markdown formatting, and red placeholder text for images were aligned correctly. I also updated the Risk Register owner columns and user personas manually to resolve the discrepancies the AI initially pointed out.

Homework 7

1. **What tool was used:**

Google Gemini

2. **What it was used for:**

I used Gemini to create a Power/Interest Grid, a Lean Six Sigma-style Project Charter, a Team Charter, format Section 1.4 external references, and fix odd formatting on the md file.

3. **The prompt(s) used:**
    - "Use the SRS as context. Create a stakeholder analysis using the power interest grid. You will also create a team charter and a project charter. Also inform me where these three items should go on the SRS template."
    - "Revise the project charter so that it follows this format. The company in the charter is not related to the one on the srs"

- "Project Charter: The Project Charter itself is typically a standalone document created before the SRS. However, within the context of the SRS, it should be heavily referenced in Section 1.4 References as an informative document guiding the platform. Alternatively, you can place the full charter text into Section 5. Appendixes. Its scope and milestone data should align with Section 1.2 Product Scope and Section 3.5.8 Deadline. Write down the reference for section 1.4"
- "Create the project Charter"
- "Create the team charter next"
- "can I remove the
 ?"
- "For the section 5 part should it be section 5.2 project charter and section 5.3 team charter?"

4. **What I kept vs. what I changed:**
    - **Verbatim / Kept:** I kept the Power/Interest Grid, Project Charter, team charter, and the Section 1.4 reference citation format generated by the AI.
    - **Modified / Changed:** I updated the Project Leader and Project Manager names to my own name (Ambar P.) instead of Alice S. I also replaced `<br>` tags with proper table formatting, and placed the Stakeholder Analysis into Section 2.4, the Project Charter into Appendix 5.2, and the Team Charter into Appendix 5.3.

5.2 Project Charter

Project Charter

Date:	**9/11/2026**
Project Title:	Smart Parking Platform (Version 6)
Project Leader:	Ambar P.
Champion:	Charlie D., Product Owner
Company / Department Description:	$oftware corp. is developing a comprehensive mobile and web-based solution to streamline parking reservations, optimize parking lot utilization, and simplify facility management.

Team Members

Team Member	Function	% Time	Initials
Ambar P.	Project Manager	50%	A.P.
Bob M.	Business Analyst	30%	B.M.
Charlie D.	Product Owner	25%	C.D.
Diana K.	UX	50%	D.K.
Evan & Fiona	Developers	100%	E.F.
George T.	Quality Assurance	50%	G.T.
Hannah P.	DevOps	30%	H.P.
Ian V.	Subject Matter Expert	10%	I.V.

Project Details

Field	Description
Process to Analyze:	The parking reservation process, from a driver locating and booking an available spot via mobile checkout, to the attendant check-in workflow using QR codes, and the automated financial settlement and dynamic pricing adjustments.
Performance Metric:	API latency for checkout endpoints (must remain under 500ms at peak load), real-time spot availability update speed, and database CPU utilization limits (under 60%) during surge pricing events.
Problem Statement:	Parking operations during major events suffer from traffic congestion and inefficient lot utilization. Drivers struggle to navigate to available parking, while facility managers are unable to maximize revenue and

Field	Description
	attendants experience bottlenecks during manual vehicle check-ins. **Who?** $oftware corp., building the platform for drivers, lot attendants, facility managers, and city planners. **What?** The end-to-end software development of the Smart Parking Platform Version 5.0, including UI/UX, database infrastructure, and API integrations.
Mission Statement:	Reduce traffic congestion during major events and maximize revenue for parking operators by providing seamless digital integration between drivers, lot attendants, and city planners. **Goal:** Implement a unified application featuring real-time spot availability, secure mobile checkout, dynamic surge pricing (triggered at 80% capacity), and digital QR-code access. **Objective:** Complete full development, user acceptance testing (UAT), and production deployment by December 18, 2026, remaining within the project budget of $232,320.
Stakeholders:	Drivers/Vehicle Owners, Facility Managers, Parking Lot Attendants, Enforcement Officers, Billing Specialists, Traffic Planners/City Officials, System Admins, and Third-Party API Providers.

5.3 Team Charter

Team Charter

Project: Smart Parking Platform (Version 6.0) **Date:** September 11, 2026

1. Team Purpose and Mission

Our mission is to successfully design, develop, and deploy the Smart Parking Platform by December 18, 2026. As a cross-functional team at $oftware corp., we will collaborate to deliver a secure, high-performing application that reduces event traffic congestion, maximizes parking operator revenue, and provides a seamless mobile checkout experience for drivers.

2. Team Members and Core Roles

Name	Role	Allocation	Core Responsibility
Ambar P.	Project Manager	50%	Schedule management, budget tracking, and risk mitigation.
Charlie D.	Product Owner	25%	Defining product vision, maximizing value, and requirement sign-off.
Bob M.	Business Analyst	30%	Eliciting requirements and translating business needs to technical tasks.
Diana K.	UX Designer	50%	Wireframing, interactive prototyping, and usability testing.
Evan & Fiona	Developers	100%	Frontend and backend implementation, database schema creation.
George T.	Quality Assurance	50%	Conducting UAT, performance/security testing, and phase-by-phase QA.

Name	Role	Allocation	Core Responsibility
Hannah P.	DevOps	30%	Cloud pipeline setup, CI/CD automation, and environment provisioning.
Ian V.	Subject Matter Expert	10%	Domain guidance on parking operations and business rules.

3. Team Norms and Communication

- **Daily Alignment:** The team will hold daily standup meetings to discuss progress, plan the day's work, and identify blockers (especially between frontend and backend integration).
- **Task Management:** All project tasks, user stories, and bugs will be tracked on the team's **Trello Scrum Board**. We will utilize persona-based color-coded labels (e.g., Green for Mobile User, Blue for Operator, Red for Backend) to maintain visual organization.
- **Quality First:** Quality Assurance is not just a final step; testing will be integrated into every project phase from design to deployment.
- **Incident Response:** Automated alerts via **PagerDuty** will be used for critical technical issues (e.g., API latency > 500ms). DevOps and Development will respond to these escalations promptly.

4. Decision Making and Conflict Resolution

- **Consensus & Ownership:** Technical decisions will be driven by the Developers and DevOps, while product scope and user value decisions will be driven by the Product Owner.
- **Conflict Resolution:** If a disagreement arises, the team will focus on data and SRS requirements to guide the solution. If an impasse is reached, the Project Manager (Alice S.) will facilitate a resolution, escalating to the Champion (Product Owner) if necessary.

5. Success Criteria

- **On-Time Delivery:** Production launch and App Store/Play Store release by the target deadline of December 18, 2026.
- **Budget Adherence:** Keep total project costs, including infrastructure and labor, within the approved budget of $232,320.
- **Technical Excellence:** Meet all Quality of Service (QoS) metrics, notably keeping checkout API latency under 500ms during peak load tests.

- **Stakeholder Satisfaction:** Achieve formal UAT sign-off from key business stakeholders and ensure all mitigation strategies for identified project risks are actively managed.